TRADERETRO

AI Copilot Report

The advisory intelligence layer of an NSE Market Data &
Backtesting Terminal for Equity, Index, Forex and Commodity assets

---

SHAURYA SINGH
BITS Pilani
August 2026

Capstone Release Candidate v0.9.0

1. Overview

The AI Copilot is an in-app advisory assistant that explains backtest results, answers domain questions and produces conversational analyses of strategy performance. It is designed as a sidecar experience: it observes the current application state (symbols, strategy configuration, backtest results, risk metrics, trade logs) and turns that context into a prompt for a configurable LLM provider. Every response is advisory only - the assistant never executes trades and has no authority over any part of the trading pipeline.

Key Capabilities

* Provider-agnostic: 13 registered models across 5 provider backends (see section 3)
* Deterministic Mock Provider fallback that works fully offline
* Six context domains injected automatically into every prompt
* Hand-crafted 7-section prompt template with chain-of-thought analysis flow
* Per-domain report templates (strategy, risk, metrics) loaded from markdown
* Client-side provider reachability probing with explicit status states
* Server-side orchestration - API keys never reach the browser
* Block-syntax markdown responses rendered in the chat panel

2. Architecture

The copilot is split across the React frontend and the FastAPI engine. In the browser, client/src/store/useAIStore.js owns the chat session: it builds a situational context string from the current backtest/strategy state, tracks the selected model, probes provider availability, and calls the backend. On the server, python-engine/ai/service.py (AIService) orchestrates three stages:

* ContextBuilder assembles the domain context (portfolio, strategy, risk, trades, market) from the request body or latest state
* PromptBuilder merges the system prompt, prompt reports and user question into a single prompt string
* AIProviderFactory resolves the requested model to a provider via the model registry and generates the response

Request Lifecycle

```
  [user question + snapshot of app state]
        |
        v
useAIStore.buildContext()  -> JSON payload (model, prompt, payload)
        |
        v
POST /api/ai/generate  ->  FastAPI router
        |
        v
AIService.generate_response()
  1. ContextBuilder.build_context(payload)
  2. PromptBuilder.build_prompt(question, context)
  3. AIProviderFactory.get_provider(model).generate_response(prompt)
```

```
        |
        v
[markdown analysis -> chat panel]
```

Timeouts are enforced at every layer: the browser uses an AbortController-based fetch wrapper (30s), and providers apply their own internal timeouts. A 503 service-unavailable result surfaces as a friendly banner in the chat panel.

3. Model Registry & Providers

python-engine/ai/registry.py defines a 13-model registry. Each entry carries an id, display name, provider backend and a local flag. ai/config.py selects the default model and provider endpoints; all local endpoints default to localhost with key-free operation for the mock/local backends.

Model ID	Display Name	Provider	Local
mock	Mock Provider	mock	yes (offline)
llama3.2	Llama 3.2	ollama	yes
llama3.1	Llama 3.1	ollama	yes
mistral	Mistral	ollama	yes
gemma2	Gemma 2	ollama	yes
gemini-3.6-flash	Gemini Flash	gemini (cloud)	no
gpt-4o-mini	GPT-4o Mini	openai (cloud)	no
qwen2.5-coder-1.5b-instruct	Qwen 2.5 Coder 1.5B	openai-compatible	yes
qwen2.5-coder-7b-instruct	Qwen 2.5 Coder 7B	openai-compatible	yes
deepseek-r1-distill-qwen-7b	DeepSeek R1 Distill 7B	openai-compatible	yes
dolphin3.0-llama3.2-3b	Dolphin 3.0 Llama 3.2 3B	openai-compatible	yes
llama-3.2-3b-instruct	Llama 3.2 3B Instruct	openai-compatible	yes
mistral-nemo-instruct	Mistral Nemo Instruct	openai-compatible	yes

Provider Backends

* mock: deterministic offline provider; always available (default in captive/demo environments)
* ollama: served by Ollama on localhost:11434 (llama3.2/3.1, mistral, gemma2)
* openai-compatible: any OpenAI-format server; the default config points at LM Studio on localhost:1234 (qwen2.5-coder 1.5B/7B, deepseek-r1, dolphin3.0)
* gemini: Google Generative Language API via GEMINI_API_KEY / GEMINI_MODEL env vars (gemini-3.6-flash)
* openai: OpenAI API services (gpt-4o-mini)
* Default model: qwen2.5-coder-1.5b-instruct (LM Studio, localhost:1234). When no local server answers, the client flags the model as unavailable and the Mock Provider remains selectable so the copilot never bricks.

Hardware Requirements (Running LM Studio)

* Minimum 8 GB high-frequency RAM required for comfortable local inference
* Minimum 4 GB VRAM-based GPU required (e.g. NVIDIA GTX 1650 / GTX 1650 Ti)

Provider Availability Probing

* Client: useAIStore.checkProviderAvailability() issues no-cors GET probes to

http://localhost:1234/v1/models and http://localhost:11434/api/tags with an AbortController timeout; result stored as providerStatus = ready | unavailable | unknown.

* Server: provider availability is scanned by the engine; the registry only exposes models whose upstream is reachable.
* The model chip in the copilot header shows the active provider; when the model is unreachable the conversation falls back with an explicit availability notice.

4. Context Builder

client/src/store/useAIStore.js builds a situational context string before every request and passes it to the backend, which validates and supplements it via python-engine/ai/context_builder.py. The context is organized into six domains, only the relevant ones being included per question:

Domain	What is injected
Market Snapshot	Selected symbols, asset classes, latest prices, market status
Portfolio	Equity curve summary, net asset value, alpha
Strategy	Active strategy (e.g. MA Crossover), SMA periods, initial capital
Risk	Risk model settings, VaR, max drawdown, exposure metrics
Trades	Recent trade log rows, PnL per instrument, exit reasons
Generic	App capabilities and available commands when no strategy is loaded

When no backtest has been run, the copilot answers from the Generic domain, so the assistant remains useful from the first message (see Fig. 3).

5. Prompt Engineering

The system prompt is hand-authored and follows a 7-section structure that anchors the assistant to its role, goals, inputs, constraints, output format, chain-of-thought analysis flow and strict response contract:

* 1. Role: platform advisor for TradeRetro, an NSE market data and backtesting terminal
* 2. Goals: explain metrics, surface risks, summarize strategies in plain language
* 3. Inputs: the injected context block (section 4), rendered as structured text
* 4. Constraints: advisory only, no trade execution, no fabricated data, no hallucinated numbers
* 5. Format: markdown with headings, bullet lists, and code blocks when relevant
* 6. Chain-of-thought: analysis steps expected for strategy/questions with numbers
* 7. Response contract: answer must cite only context values and clearly mark uncertainty

Domain-specific prompt reports live in python-engine/ai/prompts/ (strategy.md, risk.md, metrics.md) and are registered by python-engine/ai/registries/report_registry.py, so specialized deep-dive answers are appended to the main prompt when the question matches a report type.

6. API Reference

POST /api/ai/generate

```
  Request body:
{
```

```
  "model": "qwen2.5-coder-1.5b-instruct",   // registry id
  "prompt": "Explain the strategy assessment",
  "payload": { ... }                        // optional context payload
}

200 response envelope:
{
  "success": true,
  "response": "<markdown analysis>",
  "provider": "mock" | "openai-compatible" | ...
}
```

* Timeout: 30s (browser fetch wrapper aborts; providers apply internal timeouts)
* Unavailable provider: HTTP 503 + human-readable message surfaced as a chat banner
* Model unknown to the registry: falls back to the configured default model
* API keys: server-side only (gemini/openai backends); the browser never holds provider secrets
* CORS: engine allows all origins during the capstone; tighten before production

7. Availability UX

* The copilot header renders a status chip for the active provider (e.g. 'Mock Provider')
* Settings modal lists registered models and marks reachable/unreachable ones
* providerStatus drives disabled states: models on dead backends are greyed out
* Outage-style responses are detected and converted into a clear 'provider unavailable' state
* The Mock Provider is always available, keeping the demo and tests deterministic

8. Security & Limitations

* Advisory-only: the assistant cannot place orders or mutate portfolio state
* Provider keys stay in the engine environment; no key material reaches the client bundle
* All local providers run on loopback; expose none of them publicly
* The Mock Provider returns a fixed deterministic string - it is for integration/demo, not analysis
* Analysis quality depends on the local model; small models may misstate risk numbers when context is large
* Responses are grounded in the injected context only; the assistant is instructed never to invent data

9. Testing

The AI subsystem is the most heavily tested part of the platform: 225 test functions across suites covering registry consistency, provider factory resolution, context builder domain assembly, prompt template structure, report registration, service orchestration and mock/ollama/gemini provider behavior (recorded during the v0.9.0 RC audit).

* Run from python-engine: python -m pytest tests/ai -q
* All AI tests pass in CI and offline (no external service required)

10. Visual Walkthrough

Real captures of the AI Copilot at 1440x900 (Playwright Chromium, dark and light theme). In this environment no local LLM was running, so the Mock Provider served the final chat message - response text is deterministic.

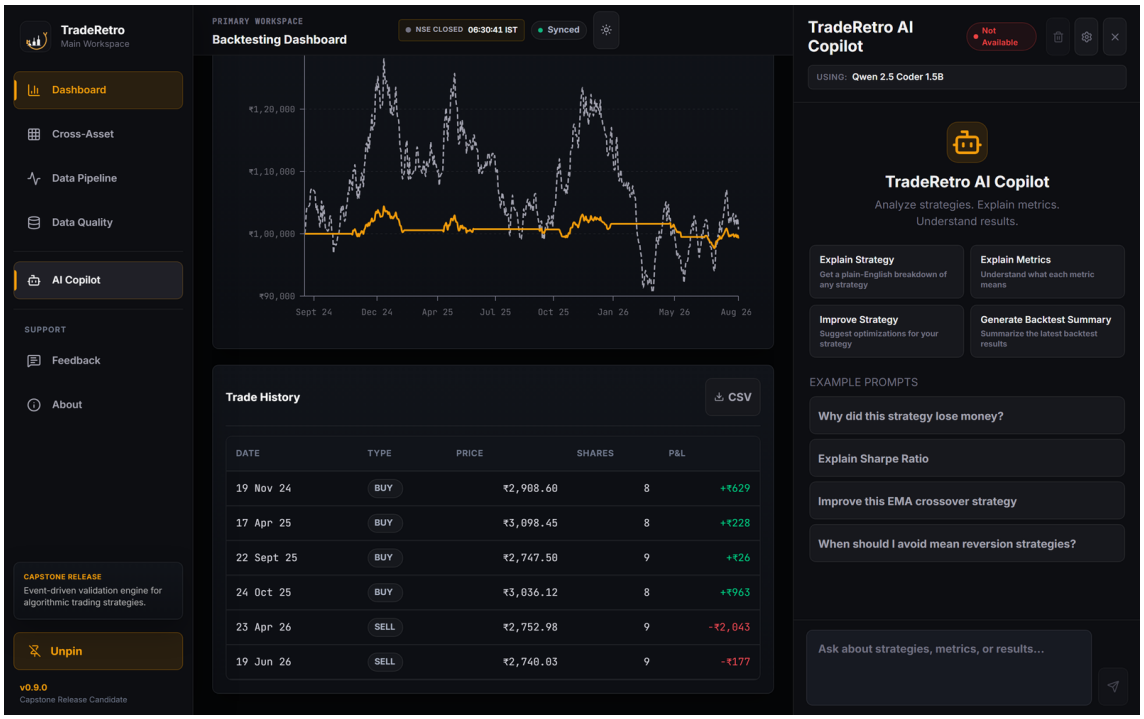

Fig. 1 - Copilot panel (dark): chat opened from the sidebar before any message.

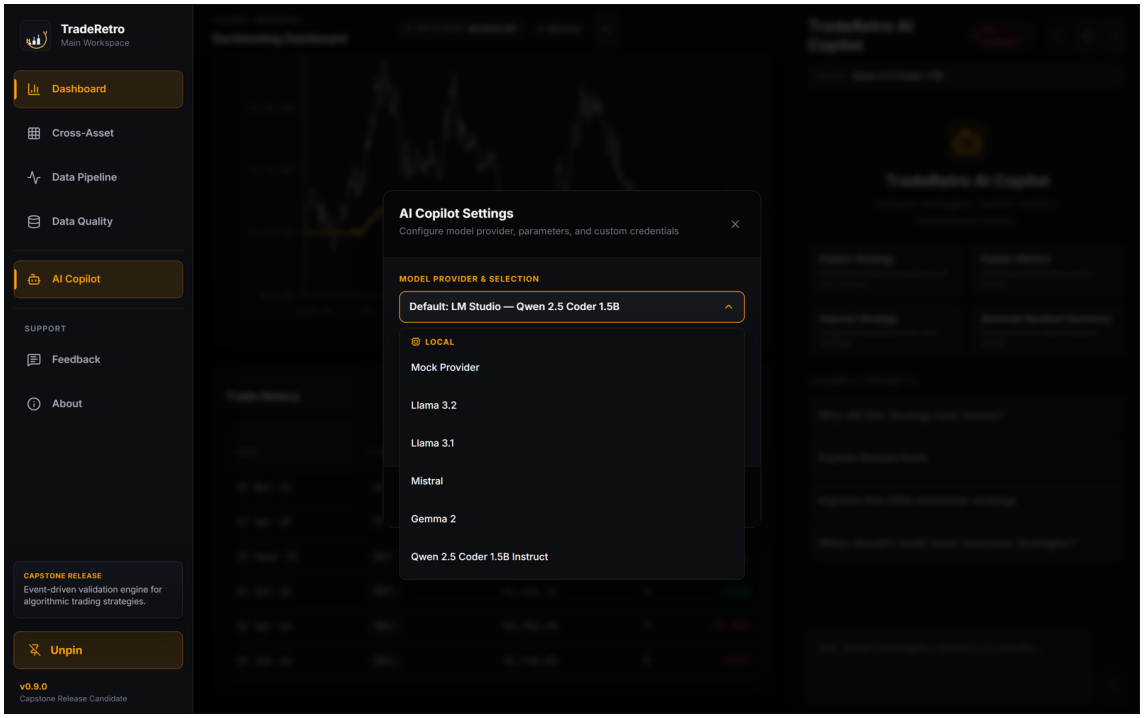

Fig. 2 - Model picker (dark): registry with availability states; Mock Provider selected.

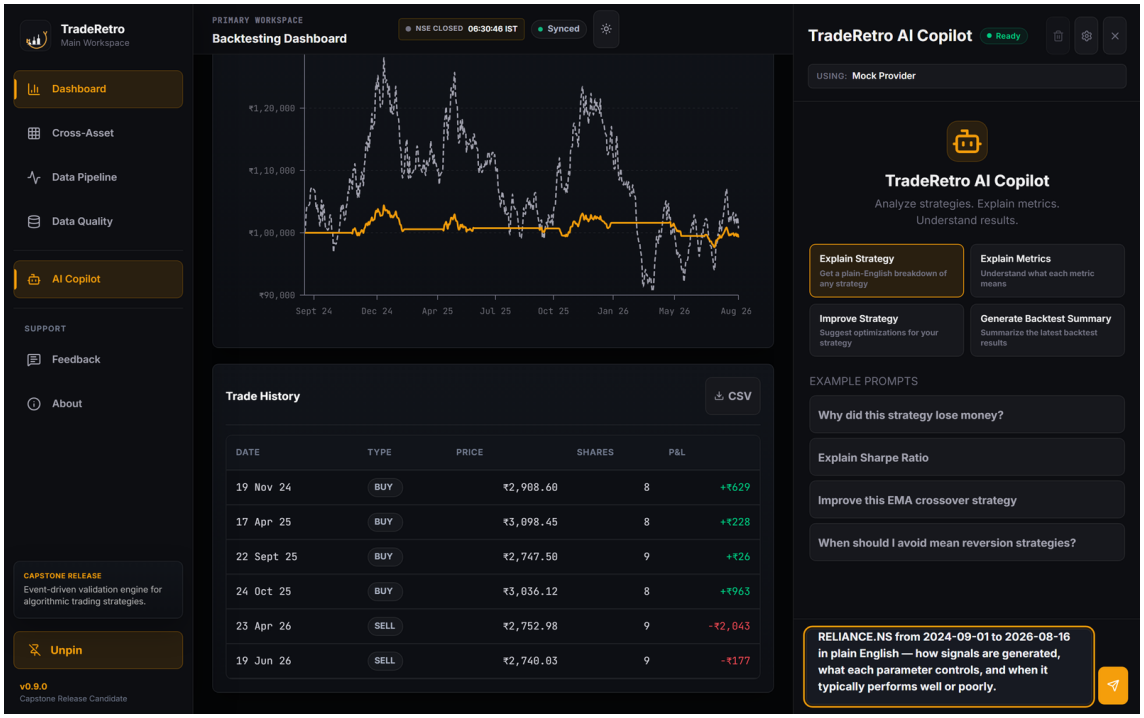

Fig. 3 - Context-aware question (dark): strategy context auto-injected from the loaded backtest.

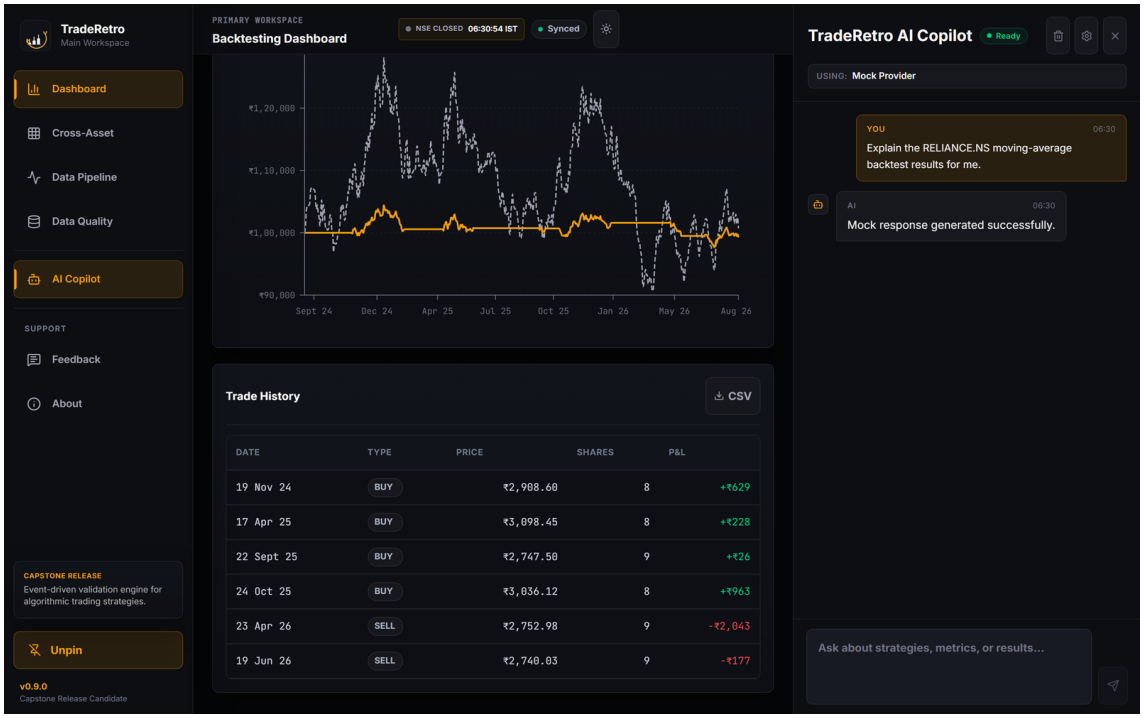

Fig. 4 - Generated analysis (dark): markdown-rendered answer with code block.

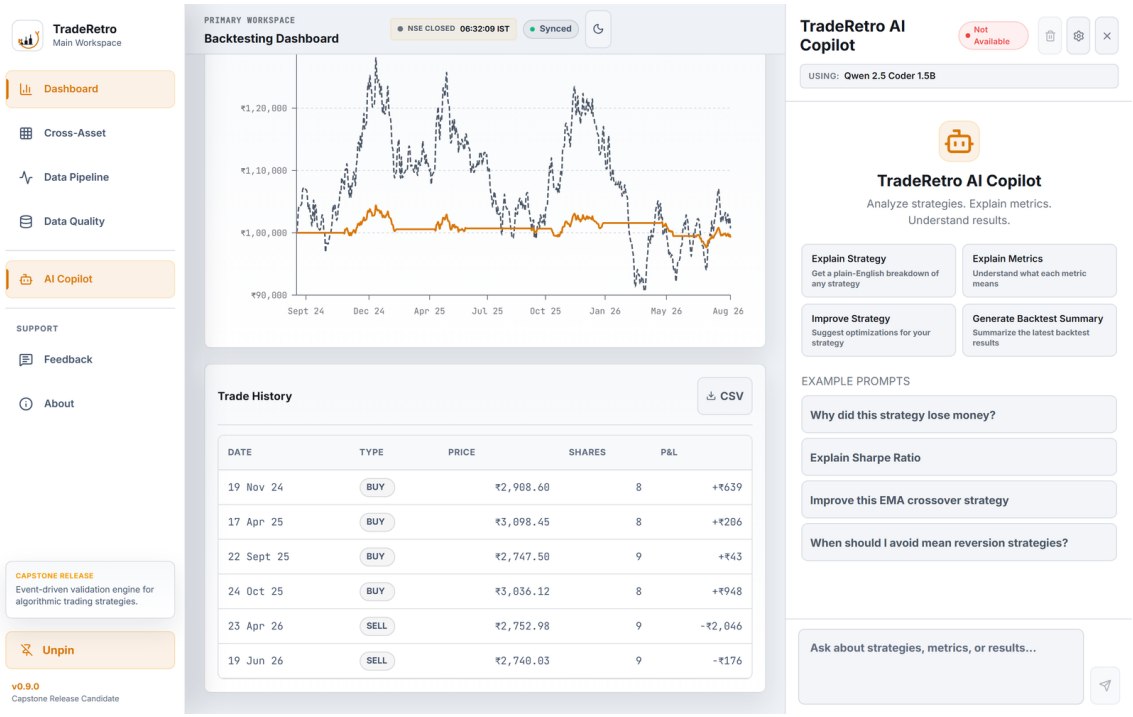

Fig. 5 - Copilot panel in the light theme.

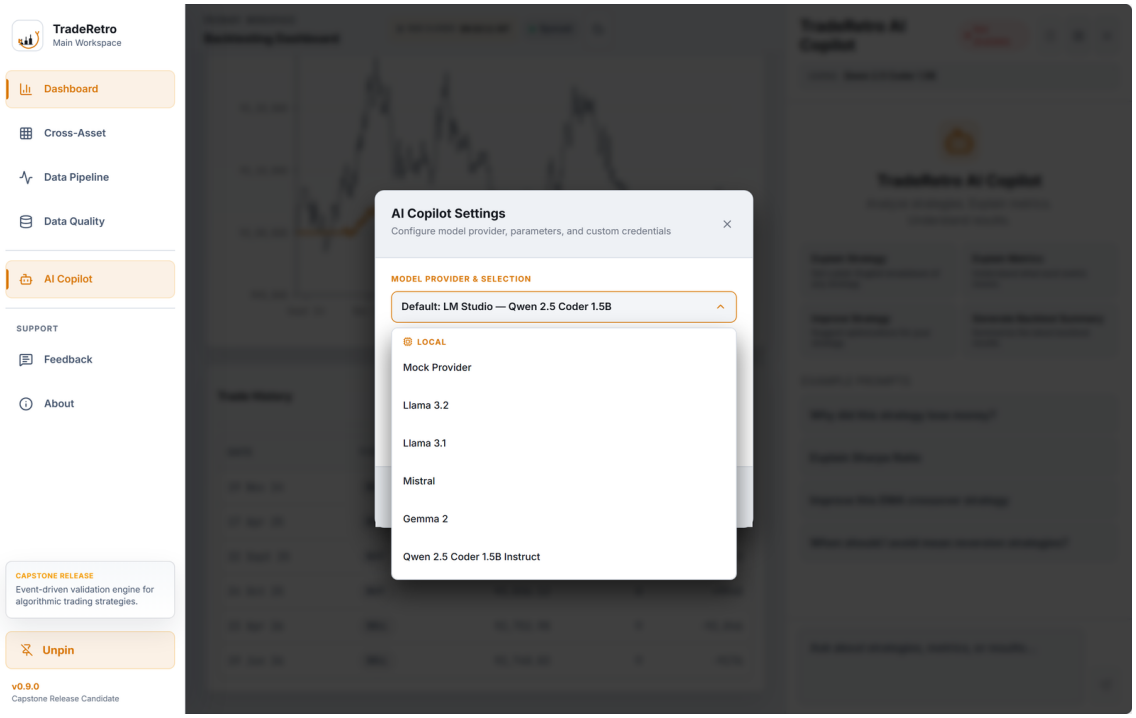

Fig. 6 - Model picker in the light theme (amber accent design system).

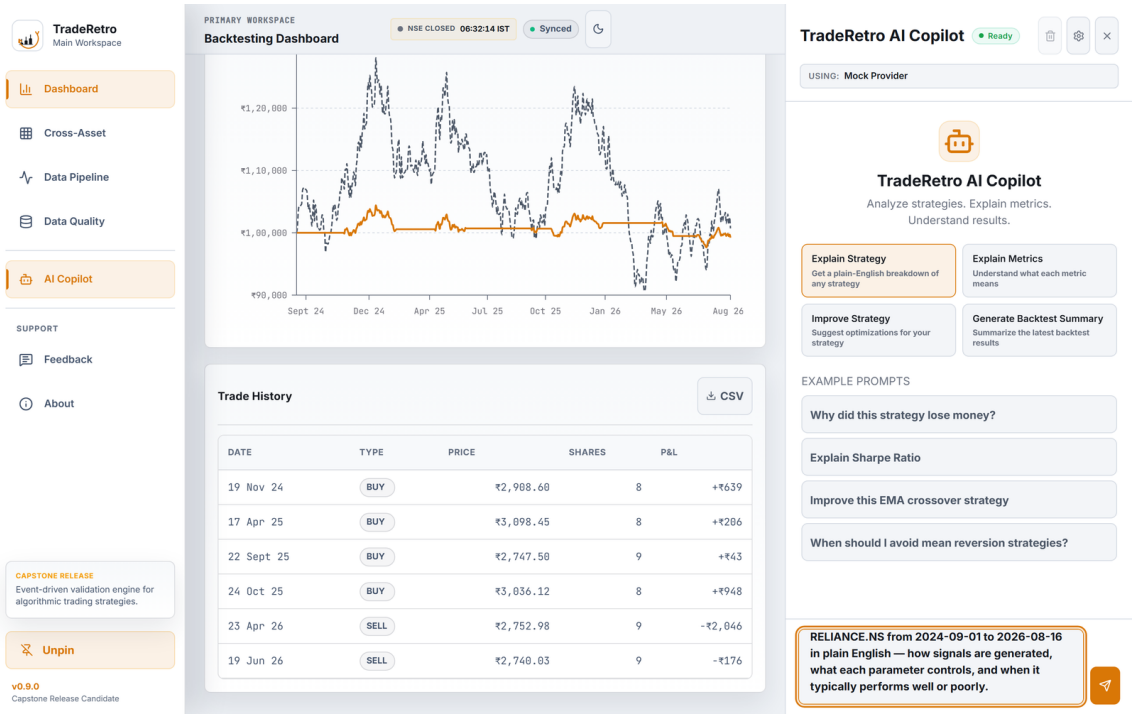

Fig. 7 - Context-autofilled question in the light theme.

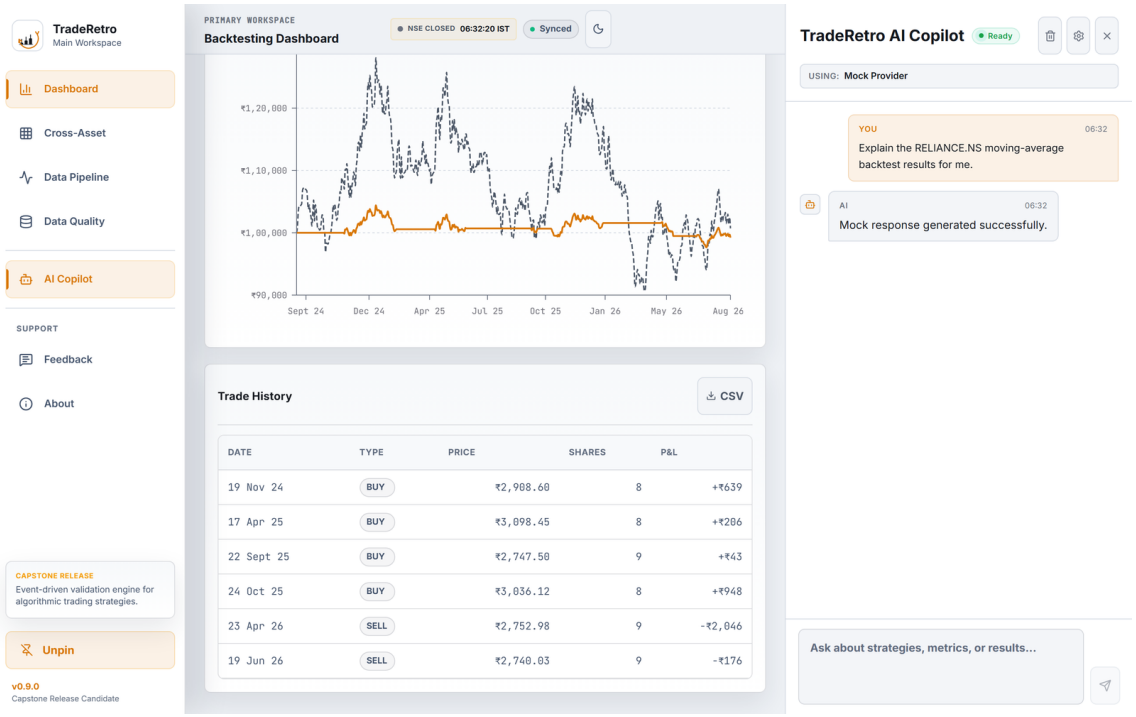

Fig. 8 - Generated analysis in the light theme.

11. Roadmap

* Streaming (token-by-token) responses instead of one-shot generation
* RAG over backtest reports with exact-number grounding
* Prompt report library expansion (per-strategy deep dives)
* Quote-level reasoning with source references in tooltips
* Usage/cost tracking per provider for cloud models